```
!pip install -q diffusers transformers accelerate
!pip install -q torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

import torch
from diffusers import StableDiffusionPipeline
import os
from PIL import Image
import matplotlib.pyplot as plt
```

```
Flax classes are deprecated and will be removed in Diffusers v1.0.0.
We recommend migrating to PyTorch classes or pinning your version of
Diffusers.
Flax classes are deprecated and will be removed in Diffusers v1.0.0.
We recommend migrating to PyTorch classes or pinning your version of
Diffusers.
```

```
# Check GPU availability
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
# Load pre-trained Stable Diffusion model
model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16
)

pipe = pipe.to(device)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```


```
{"model_id":"c8bfa3e69efb45d89c54084b7b51d3e2","version_major":2,"vers
ion_minor":0}

{"model_id":"f69fbd50fd844ea3835c904154e6a8d2","version_major":2,"vers
ion_minor":0}

{"model_id":"90c25c1ad5504fb1b17bfaf1dbee3d83","version_major":2,"vers
ion_minor":0}
```

{"model_id":"2135bf4086c54df6b67414d2dfa0bf72","version_major":2,"version_minor":0}

{"model_id":"63bd180c164c4c9b8fe217515b81bd2f","version_major":2,"version_minor":0}

{"model_id":"baa8ac9dde99424aa5889ccc5f71d594","version_major":2,"version_minor":0}

{"model_id":"4504ba0737f44d6e8c826ecac8d6b4c1","version_major":2,"version_minor":0}

{"model_id":"acdd8a3bdd0642e39b8a0f02ae56d1dd","version_major":2,"version_minor":0}

{"model_id":"7711846d984f4f36b85266b84f63754d","version_major":2,"version_minor":0}

{"model_id":"4cc08f4206764b7293b450c67aff2c6e","version_major":2,"version_minor":0}

{"model_id":"66e4836dc730455eac9c21f1530f60d5","version_major":2,"version_minor":0}

{"model_id":"451628daabc541cfae941bdc004b6ca3","version_major":2,"version_minor":0}

{"model_id":"d82921d074af43f4aa06661f4e26ec15","version_major":2,"version_minor":0}

{"model_id":"b812f182cfd74699b5d3386f056b4d1d","version_major":2,"version_minor":0}

{"model_id":"c467166fcc964cfd87ad2cea768e5f42","version_major":2,"version_minor":0}

{"model_id":"c8eaf39fd6f74496aaf8f2c7988a383b","version_major":2,"version_minor":0}

{"model_id":"1fc847d46d6d408fb62159373e6d7a33","version_major":2,"version_minor":0}

`torch_dtype` is deprecated! Use `dtype` instead!

```
# Create output dataset directory
output_dir = "synthetic_image_dataset"
os.makedirs(output_dir, exist_ok=True)

# Input prompts (minimum 5)
prompts = [
    "A futuristic smart city at night",
    "A realistic portrait of Virat Kohli wearing Indian cricket team jersey, professional sports photography, stadium background, high
```

```
detail, ultra realistic lighting",
    "A beautiful sunset over the mountains",
    "A dynamic action shot of Chennai Super Kings players celebrating
a victory in IPL, yellow jerseys, packed cricket stadium, cinematic
lighting",
    "A close-up view of a colorful flower blooming in a natural
garden, soft sunlight, detailed petals, macro photography, ultra
realistic, high resolution"
]

# Generate and save images
for i, prompt in enumerate(prompts):
    print(f"Generating image {i+1}...")
    image = pipe(prompt).images[0]
    image.save(f"{output_dir}/image_{i+1}.png")

print("Synthetic image dataset generated successfully!")
```

```
Generating image 1...
```


```
{"model_id":"064984e4a1b14596932313a58282e997","version_major":2,"vers
ion_minor":0}
```


```
Generating image 2...
```


```
{"model_id":"7b766bd0ece34502b1279b2b8ef4dcb6","version_major":2,"vers
ion_minor":0}
```


```
Generating image 3...
```


```
{"model_id":"b611305858f24987a42c165eded67810","version_major":2,"vers
ion_minor":0}
```


```
Generating image 4...
```


```
{"model_id":"25e32749894f4aeeabd948b50984c6f4","version_major":2,"vers
ion_minor":0}
```


```
Generating image 5...
```


```
{"model_id":"23d02c488c97450fbba48ce9b8210d4e","version_major":2,"vers
ion_minor":0}
```


```
Synthetic image dataset generated successfully!
```

```
# Folder containing generated images
image_folder = "synthetic_image_dataset"

# Get all image file names (sorted)
image_files = sorted(os.listdir(image_folder))[:5]

# Create a figure to display images
```

```
plt.figure(figsize=(15, 6))

for i, img_name in enumerate(image_files):
    img_path = os.path.join(image_folder, img_name)
    img = Image.open(img_path)

    plt.subplot(1, 5, i + 1)
    plt.imshow(img)
    plt.title(f"Image {i + 1}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```